



Motion Control Library in “C++”

Controlling Drives based on CiA402 device profile

- EC-Motion at a glance
- Introduction & Standards
- EC-Motion software architecture
- Function Blocks
 - Administrative
 - Single-Axis Motion
 - Multi-Axis Motion
- Examples
- Highlights

- Motion Control library and example application for operating servo drives based on CiA402 drive profile
- Single-Axis and Multi-Axis (Camming and Gearing) movements.
Limitations: No coordinated motion, no robotics functions, no CNC functions.
- Sophisticated trajectory generator
 - Jerk-limited movements
 - Sequence of movements without stopping → Buffered Modes
- Provides a Programmable Logic Controller (PLC) style interface.
It is designed to be easy integrating in a PLC for controlling servo drives.
- Number of supported drives: 32
- Typical applications:
 - Positioning, feeding, pick-and-place
 - Assembly Lines, packaging, tool changer, etc.

Motion Control Functions

The CiA402 profile makes it's possible to operate drives from different manufacturers without specific implementation on the application level.

MainDevice

EtherCAT Technology Group ETG.6010 Implementation Directive for CiA402 Drive Profile

SubDevice

Additional implementation hints for using CiA402 with EtherCAT technology

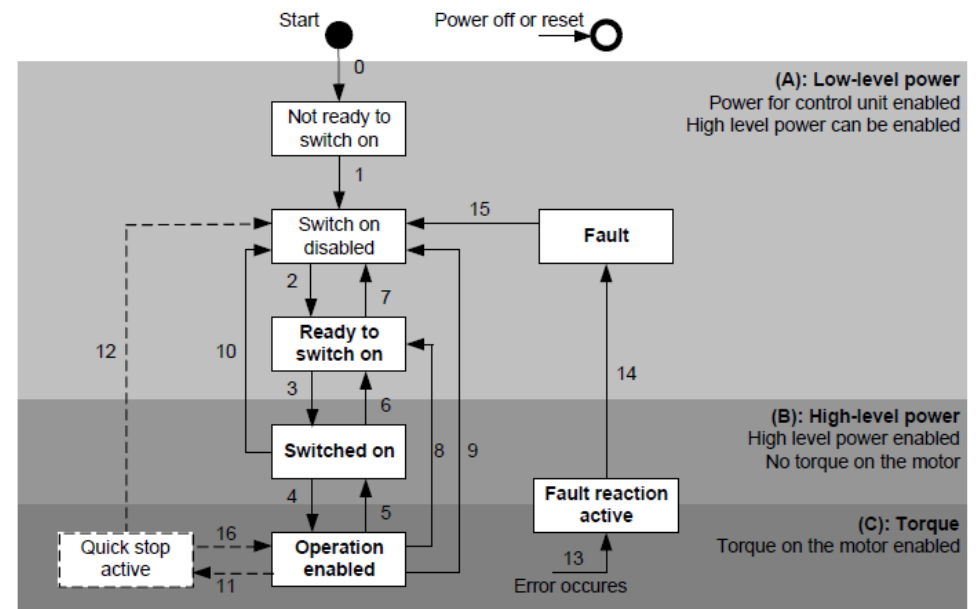
CiA402: CANopen device profile for drives and motion control

CiA402 organizes parameters in a so-called object dictionary and a drive state machine → drive profile

Most available drives with EtherCAT interface are based on the CiA402 drive profile, e. g., Yaskawa, Copley, Beckhoff, Omron, ...

CiA402: CANopen device profile for drives and motion control

- CiA402 organizes parameters in a so called object dictionary. Each parameter has a defined number (index + subindex) and meaning
 - Object 0x6040: Control Word
 - Object 0x6041: Status Word
 - Object 0x607A: Target Position
 - Object 0x6064: Actual Position
 -
- CiA402 drive state machine



ETG Implementation Directive for CiA402 Drive Profile

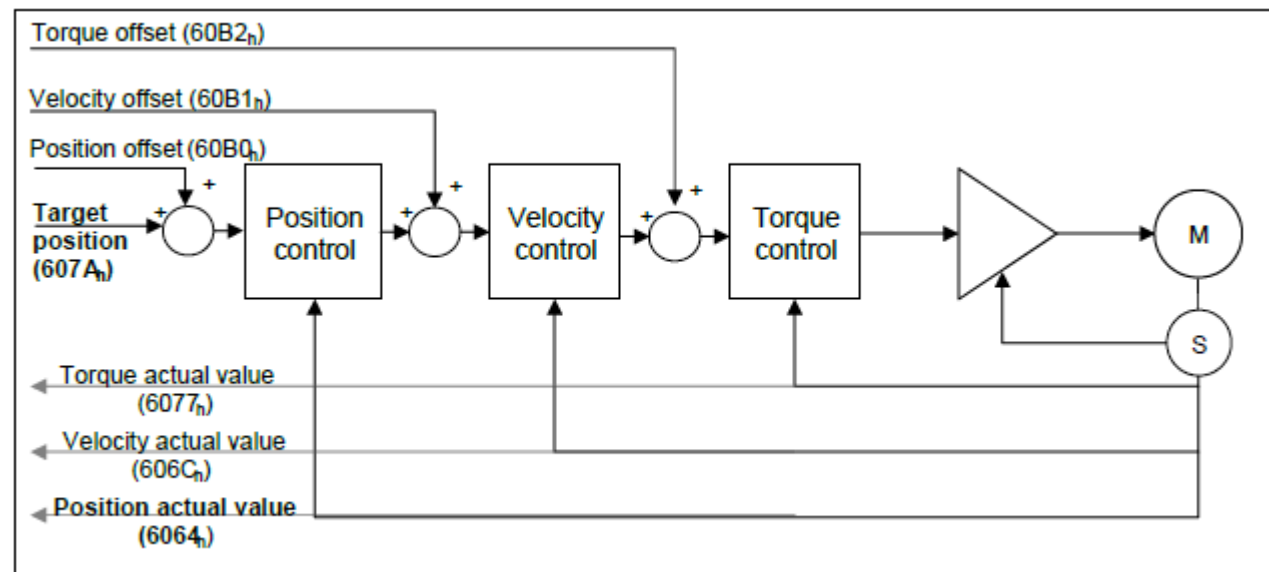
- The drive has to support at least one of the cyclic operation modes:
CSP or CSV or CST

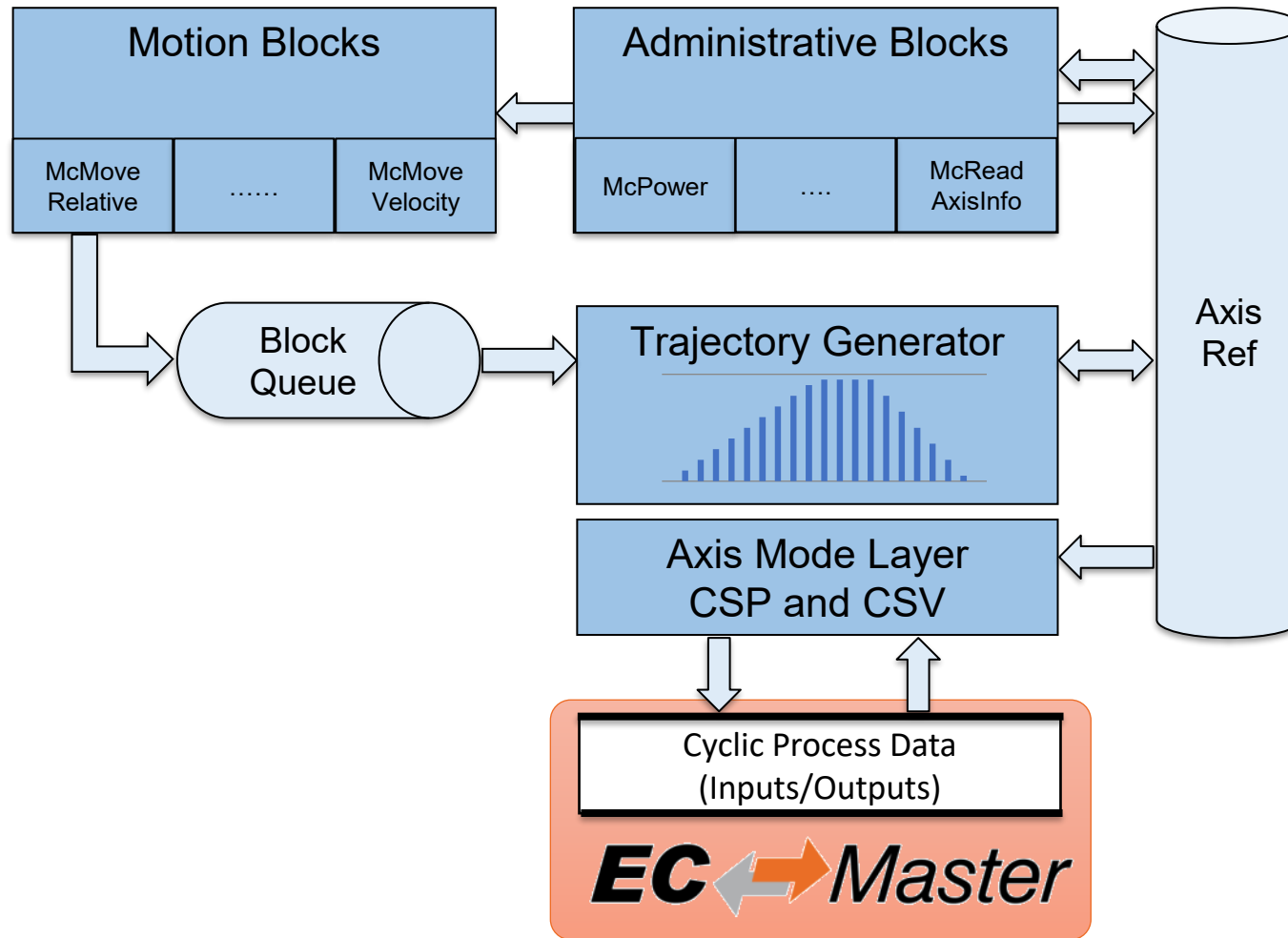
Mode of operation	Abbr.	Code	Category	Remarks
Profile position mode	pp	1	O	
Velocity mode (frequency converter)	vl	2	O	
Profile velocity mode	pv	3	O	
Torque profile mode	tq	4	O	
Homing mode	hm	6	O	
Interpolated position mode	ip	7	O	
Cyclic synchronous position mode	csp	8	C	at least one of these modes shall be supported
Cyclic synchronous velocity mode	csv	9	C	
Cyclic synchronous torque mode	cst	10	C	
Cyclic synchronous torque mode with commutation angle	cstca	11	O	
Manufacturer specific mode		-128...-1	O	

ETG Implementation Directive for CiA 402 Drive Profile

CSP: Cyclic Synchronous Position Mode

- Application has to set a new “Target position” in every cycle (trajectory generator)
- Position, Velocity and Torque are controlled by the drive





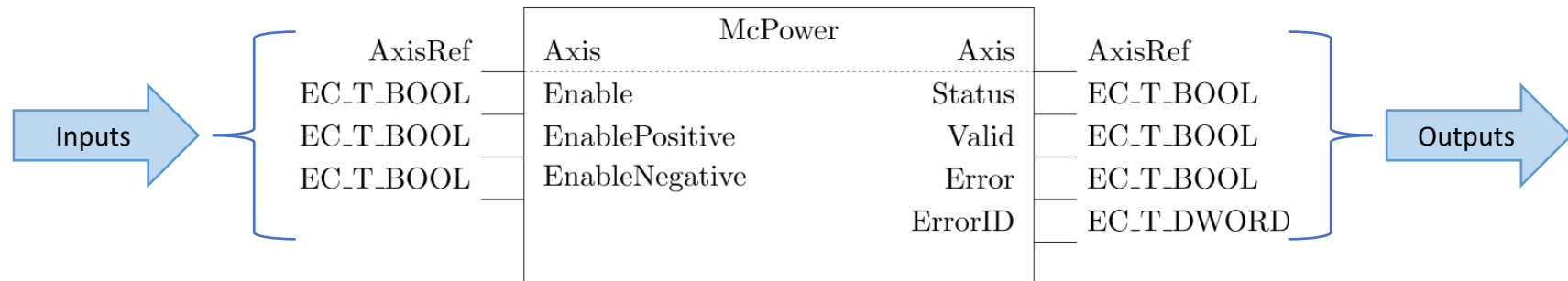
EC  ***Motion***

Administrative Function Blocks

Administrative Function Block

McPower: Function Block diagram

This Function Block controls the power stage (On or Off).



VAR_INOUT				
	B	Axis	AxisRef	Reference to the axis
VAR_IN				
	B	Enable	EC_T_BOOL	As long as 'Enable' is true, power is being enabled.
	E	EnablePositive	EC_T_BOOL	As long as 'Enable' is true, this permits motion in positive direction.
	E	EnableNegative	EC_T_BOOL	As long as 'Enable' is true, this permits motion in negative direction.
VAR_OUT				
	B	Status	EC_T_BOOL	Effective state of the power stage.
	E	Valid	EC_T_BOOL	If true, a valid set of outputs is available at the FB.
	B	Error	EC_T_BOOL	Signals that an error has occurred within the function block.
	E	ErrorID	EC_T_DWORD	Error identification.

Administrative Function Block

McPower: C++ API

```
class McPower
{
public:
    /* IN-OUT's */
    VAR_INOUT<AxisRef>::type Axis;                /* INOUT(B): Reference to axis */

    /* OUT's */
    VAR_OUT<EC_T_BOOL>::type Status;               /* OUT(B): Effective state of the power stage */
    VAR_OUT<EC_T_BOOL>::type Valid;                /* OUT(E): If TRUE a valid set of outputs is available */

    /* IN's */
    VAR_IN<EC_T_BOOL>::type  Enable;                /* IN(B): As long as is true, power is on */
    VAR_IN<EC_T_BOOL>::type  EnablePositive;         /* IN(E): As long as is true, permits motion in pos direction only */
    VAR_IN<EC_T_BOOL>::type  EnableNegative;         /* IN(E): As long as is true, permits motion in neg direction only */

    EC_T_VOID operator () ();
};
```

Administrative Function Block

McPower: Example Code Snippet

```
/* axis initialization */
{
    MC_T_AXIS_PARMS oAxisParms;
    AxisRef*        pMcAxis;

    OsMemset(&oAxisParms, 0, sizeof(MC_T_AXIS_PARMS));
    oAxisParms.dwCycleTime = 1000;
    oAxisParms.dwIncPerMM = 10000;

    mcCreateVirtualAxis(oAxisParms, pMcAxis);
}

/* cyclic application */
{
    McPower oMcPower;
    oMcPower.Axis = pMcAxis;

    while (EC_TRUE)
    {
        oMcPower.Enable = EC_TRUE;
        oMcPower();

        /* further application like movements etc. */
    }
}
```

Administrative Functions Blocks Summary

- **McPower:** This Function Block controls the power stage (On or Off).
- **McSetPosition:** This function block shifts the coordinate system of an axis by manipulating both the set-point position as well as the actual position of an axis with the same value without any movement caused.
- **McReadParameter, McReadBoolParameter:** Returns the value of a parameter.
- **McWriteParameter, McWriteBoolParameter:** Modifies the value of a parameter.
- **McReadDigitalInput, McReadDigitalOutput, McWriteDigitalOutput:** Function Blocks gives access to the value of the input and outputs.
- **McReadActualPosition:** This Function Block returns the actual position.
- **McReadStatus:** This function block returns in detail the status of the state diagram of the selected axis.
- **McReadAxisInfo:** This Function Block reads information concerning an axis.
- **McReset:** This function block makes the transition from the state 'ErrorStop' to 'Standstill' or 'Disabled' by resetting all internal axis-related errors - it does not affect the output of the FB instances.

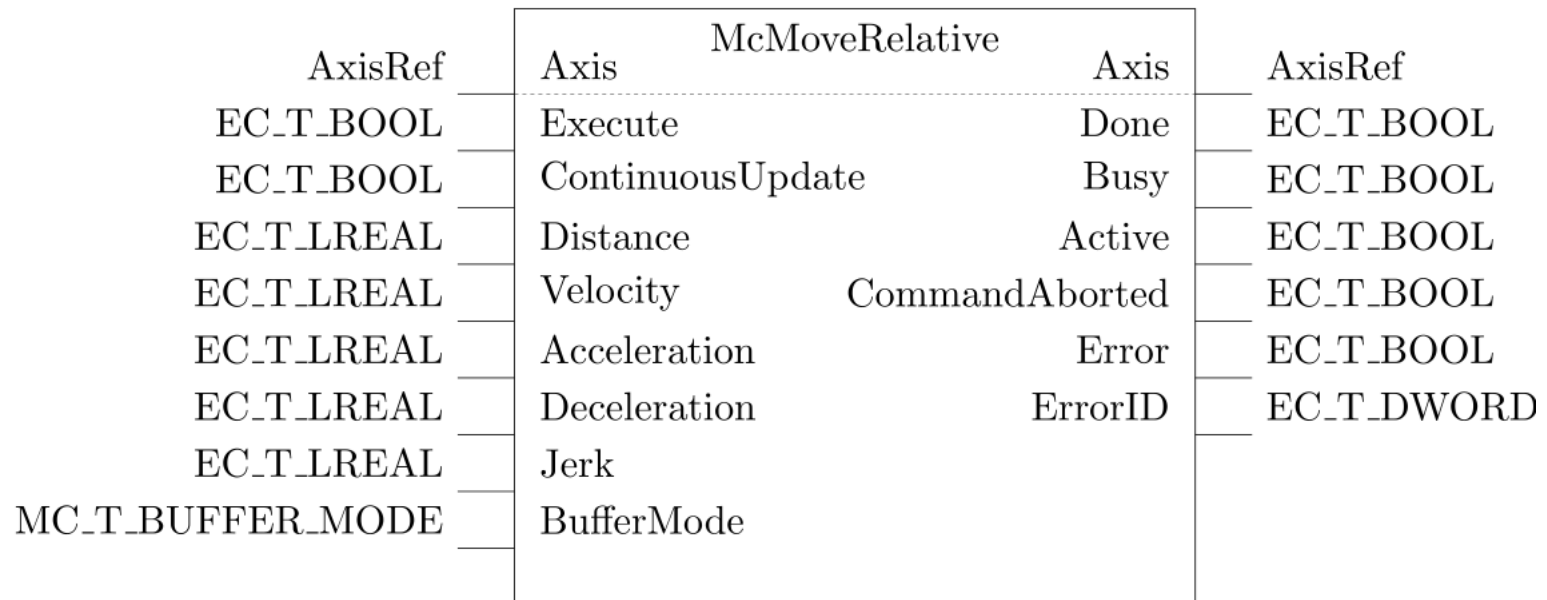
EC ***Motion***

Single-Axis Motion Function Blocks

Single-Axis Motion Function

McMoveRelative: Function Block Diagram

Commands a controlled motion of a specified distance relative to the set position at the time of the execution.



Single-Axis Motion Function

McMoveRelative: C++ API

```
class McPower
{
public:
    /* IN-OUT's */
    VAR_INOUT<AxisRef>::type Axis;          /* INOUT(B): Reference to axis */

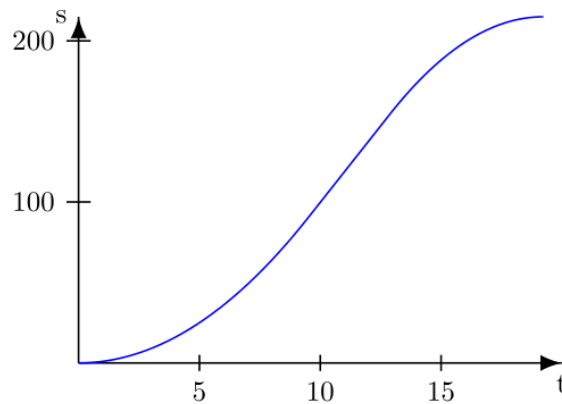
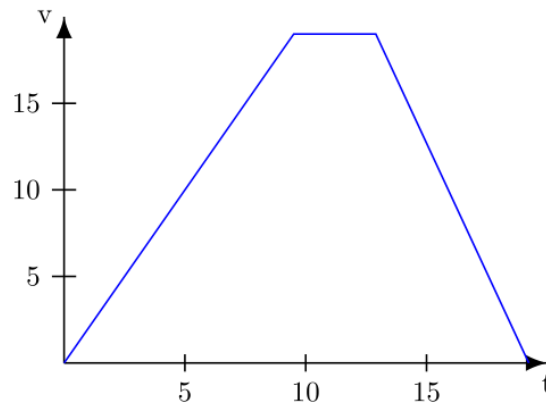
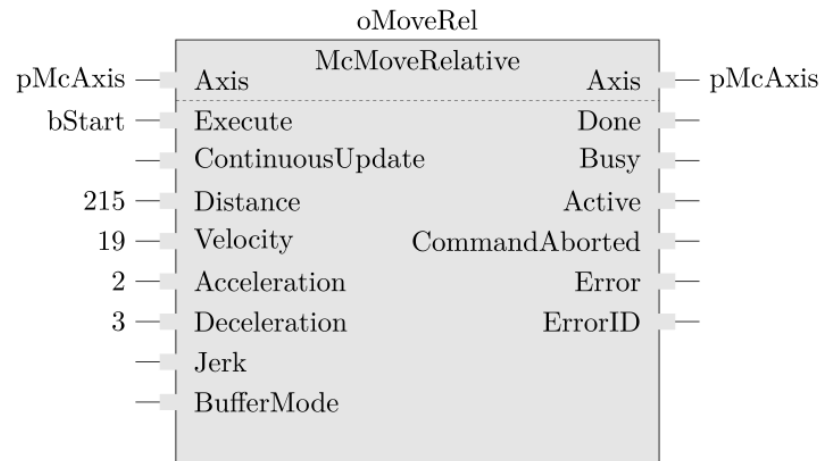
    /* OUT's */
    VAR_OUT<EC_T_BOOL>::type Done;          /* OUT(B): Commanded distance reached */
    VAR_OUT<EC_T_BOOL>::type Busy;          /* OUT(E): Not finished and new output to be expected */
    VAR_OUT<EC_T_BOOL>::type Active;        /* OUT(E): Function block has control on the axis */
    VAR_OUT<EC_T_BOOL>::type CommandAborted; /* OUT(E): Command is aborted by another command */
    VAR_OUT<EC_T_BOOL>::type Error;         /* OUT(B): Error has occurred within function block */
    VAR_OUT<EC_T_DWORD>::type ErrorID;      /* OUT(E): Error identification */

    /* IN's */
    VAR_IN<EC_T_BOOL>::type Execute;        /* IN(B): Start movement at rising edge */
    VAR_IN<EC_T_BOOL>::type ContinuousUpdate; /* IN(E): Apply input values to ongoing movement */
    VAR_IN<EC_T_LREAL>::type Distance;      /* IN(B): Relative distance for movement */
    VAR_IN<EC_T_LREAL>::type Velocity;      /* IN(E): Value of maximal velocity */
    VAR_IN<EC_T_LREAL>::type Acceleration;   /* IN(E): Value of acceleration */
    VAR_IN<EC_T_LREAL>::type Deceleration;  /* IN(E): Value of deceleration */
    VAR_IN<EC_T_LREAL>::type Jerk;          /* IN(E): Value of jerk */
    VAR_IN<MC_T_BUFFER_MODE>::type BufferMode; /* IN(E): Define chronological sequence of function block */

    EC_T_VOID operator () ();
};
```


Single-Axis Motion Function

McMoveRelative: Example (1)



Single-Axis Motion Function

McMoveRelative: Example (2)

```
/* parametrization of function block */
{
    oMoveRel.Axis = pMcAxis;
    oMoveRel.Distance = 215;
    oMoveRel.Velocity = 19;
    oMoveRel.Acceleration = 2;
    oMoveRel.Deceleration = 3;
}

/* cyclic application */
bStart = EC_FALSE;
while (EC_TRUE)
{
    mcRefreshAxisInputs(0, 0, pbyPdIn);

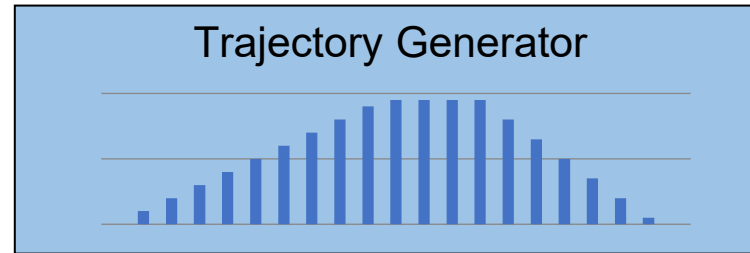
    if (/* external signal or another input */)
    {
        bStart = EC_TRUE;
    }
    oMoveRel.Execute = bStart;
    oMoveRel();

    mcCyclicTask(0, 0);
    mcRefreshAxisOutputs(0, 0, pbyPdOut);
}
```

- **McStop:** Commands a controlled motion stop and transfers the axis to the state 'Stopping'.
- **McHalt:** Commands a controlled motion stop and transfers the axis to the state 'Standstill'.
- **McMoveAbsolute:** Commands a controlled motion to a specified absolute position.
- **McMoveRelative:** Commands a controlled motion of a specified distance relative to the set position at the time of the execution.
- **McMoveVelocity:** Commands a never-ending controlled motion at a specified velocity.
- **McMoveContinuousAbsolute:** Commands a controlled motion to a specified absolute position ending with the specified velocity.
- **McMoveContinuousRelative:** Commands a controlled motion of a specified relative distance ending with the specified velocity.

EC  ***Motion***

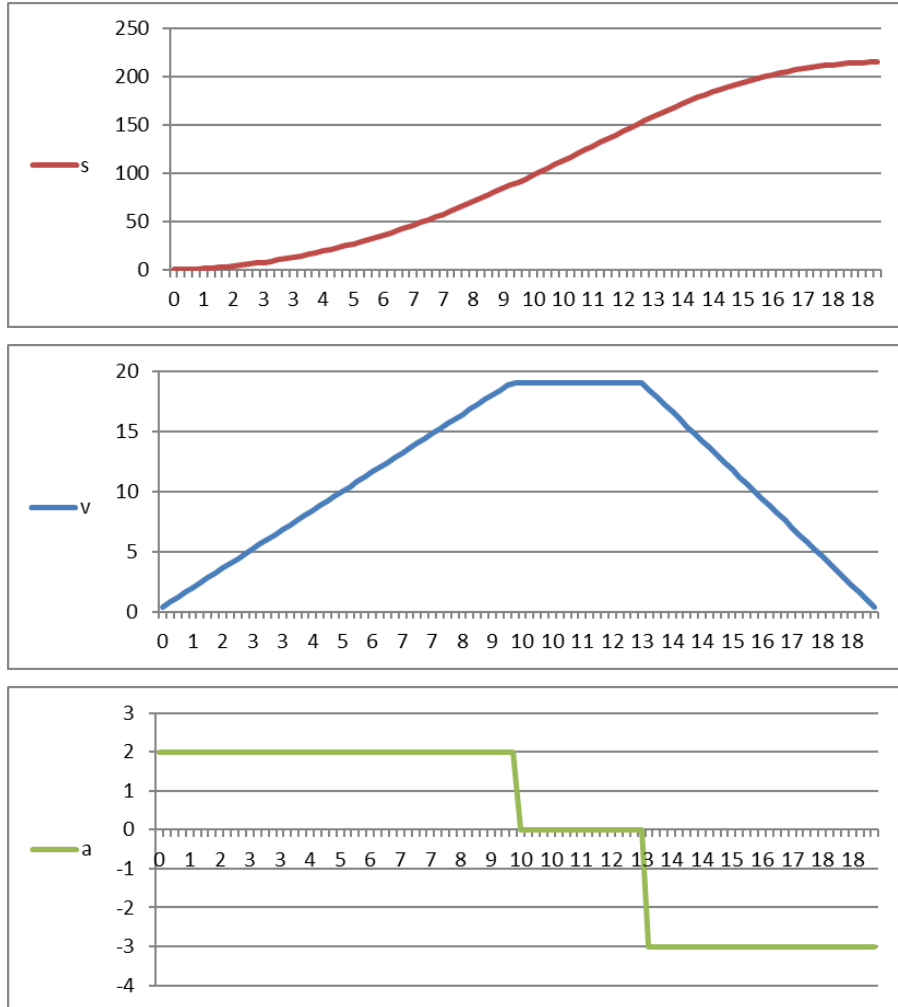
Trajector Generator



- Based on the given dynamic of motion command (maximal velocity, acceleration, deceleration and jerk) determine velocity profile of movement to desired position
- With specified cycle time interpolate velocity profile to computed commanded position resp. velocity for each cycle

Trajectory Generator

Example: Continuous motion

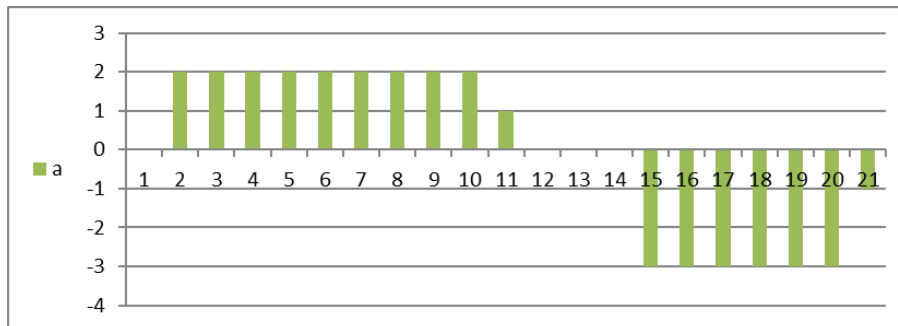
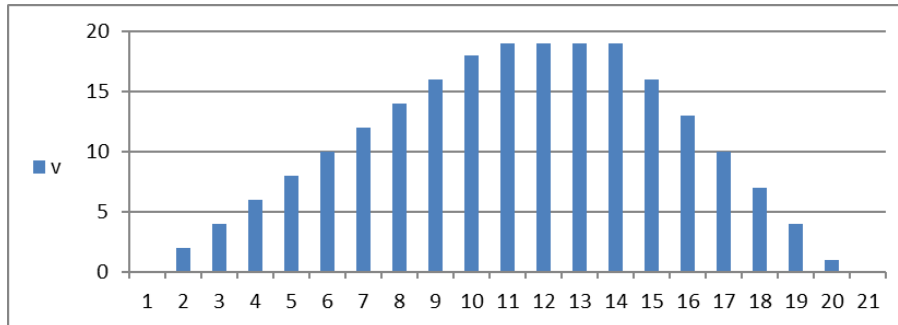
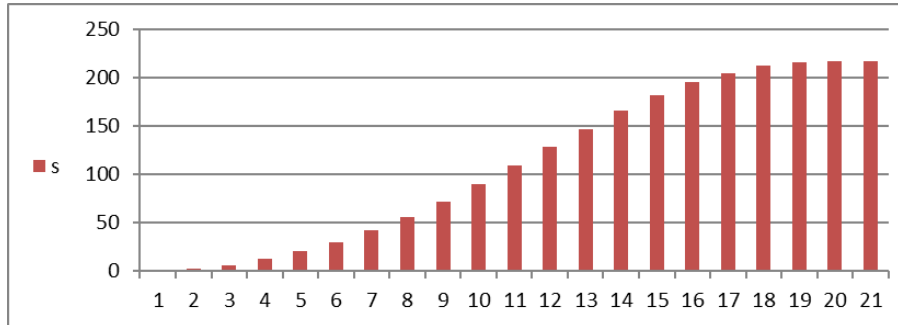


Constantly accelerated and deaccelerated movement with a maximum velocity:

- distance $s=217$
- max. velocity $v=19$
- accelerate $a=2$
- decelerate $a=-3$

Trajectory Generator

Example: Discrete motion



Same movement:

- Distance $s=217$, max. Velocity $v=19$
- Accelerate $a=2$, Decelerate $a=-3$

Split into three sections:

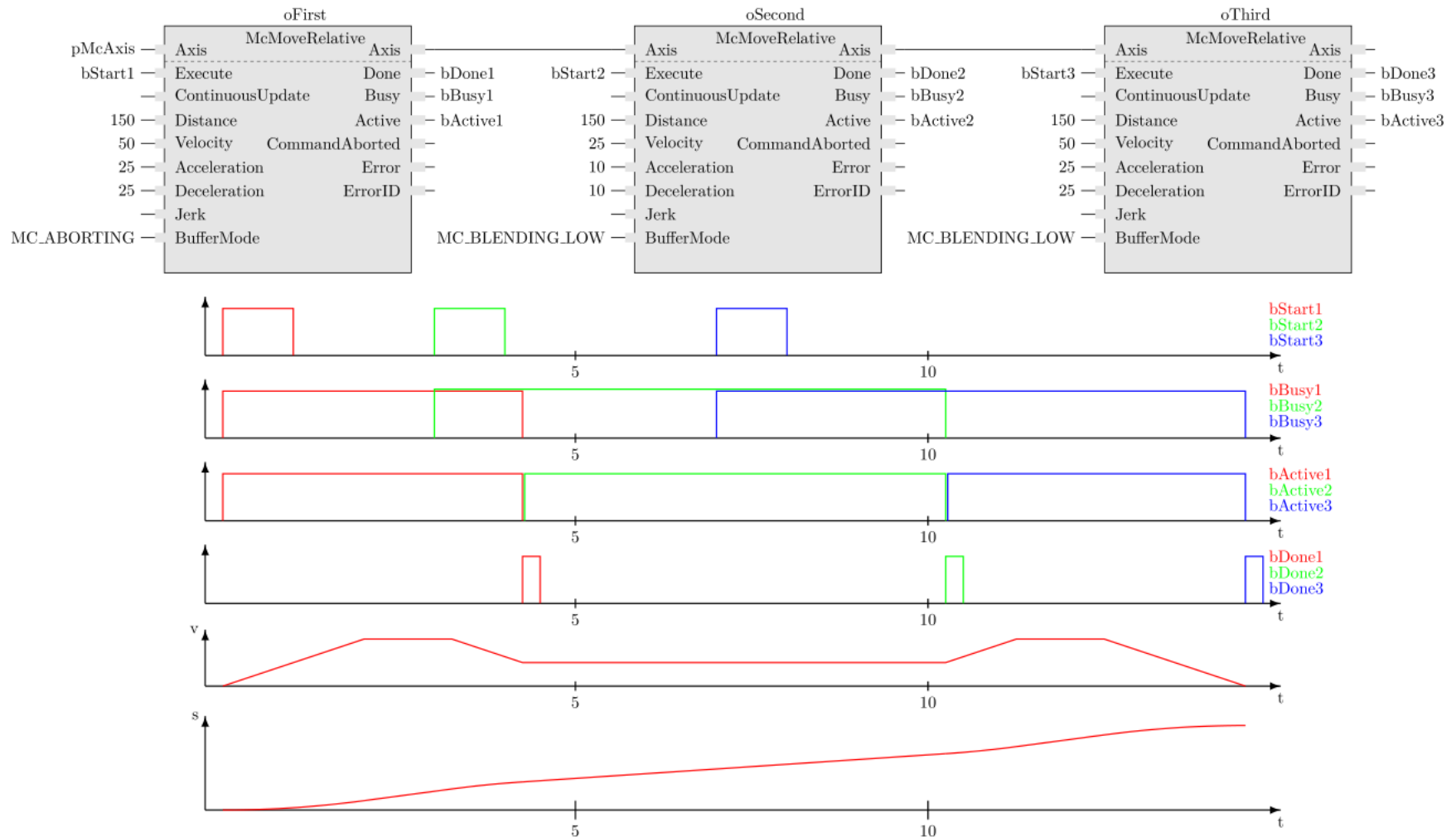
- Section 1: Accelerate
 $a=2$ up to $v=19 \rightarrow s=90$
- Section 2: Constant Velocity
 $v=19 \rightarrow s=76$
- Section 3: Decelerate
 $a=-3$ down to $v=0 \rightarrow s=51$

EC ***Motion***

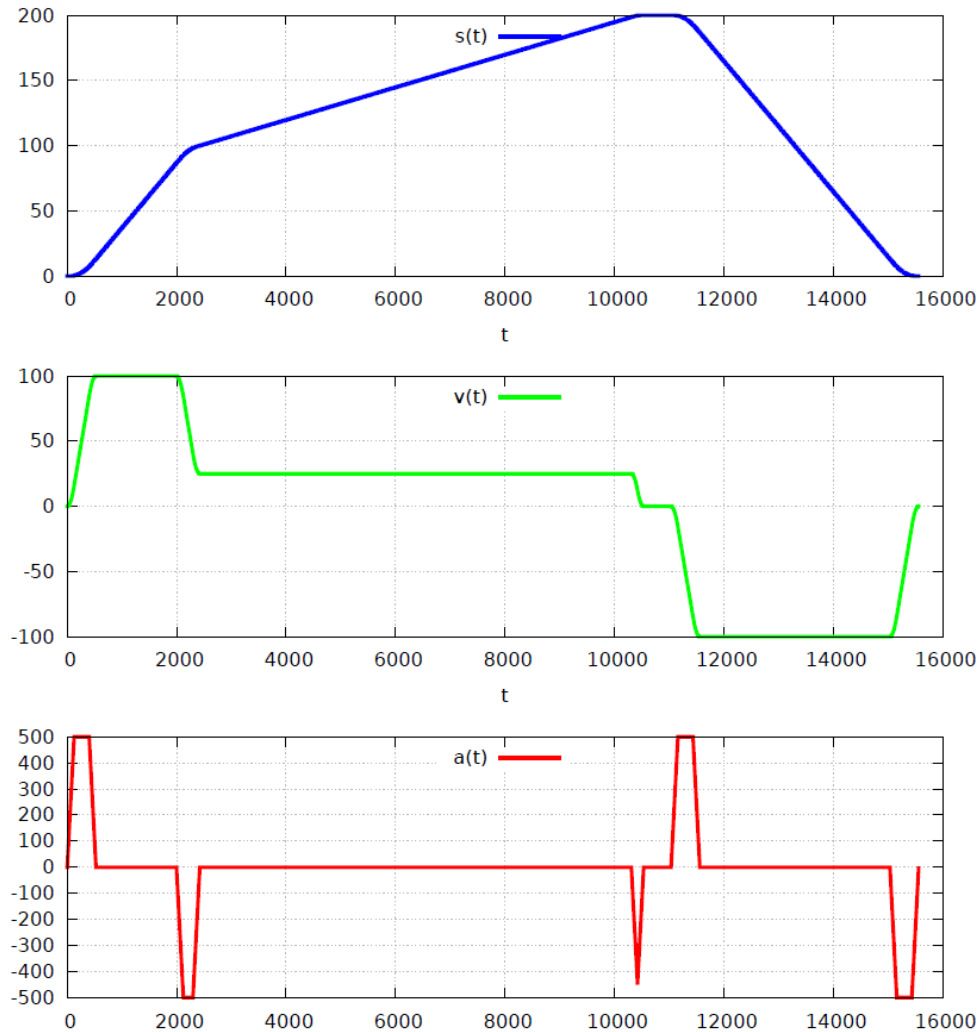
Advanced Single-Axis Features

- Link movements together using buffering:
 - A command in a non-Buffered Mode acts immediately, even if this interrupts another movement.
 - A command in a Buffered Mode waits till the current FB sets its 'Done' output (or 'InPosition', or 'InVelocity',...)
- Buffered Modes:
 - Aborting: Default mode without buffering
 - Buffered: The next FB affects the axis as soon as the previous movement is 'Done'. There is no blending.
 - BlendingLow: The next FB controls the axis after the previous FB has finished (equivalent to 'Buffered'), but the axis will not stop between the movements. The velocity is blended with the lowest velocity of both commands (1 and 2) at the first end-position (1)
 - Other modes: BlendingPrevious, BlendingNext, BlendingHigh

Example 1: Buffermode = MC_BLENDING_LOW



Example 2: Drill a hole using mcBlendingLow and mcAborting

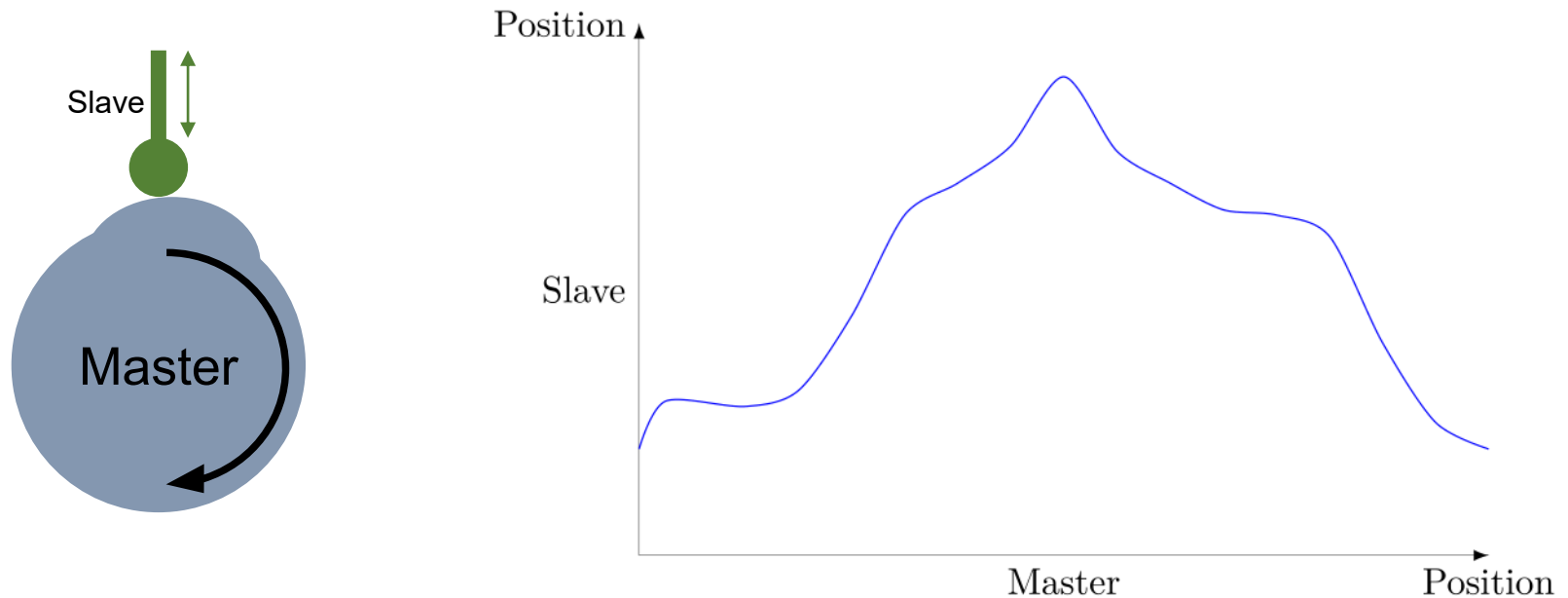


EC ***Motion***

Multi-Axis Motion Function Blocks

- Multi-Axis Function Blocks establish a relationship between a master axis and one or more slave axes
- Master axis can be a virtual axis
- Slave axis is in a specific synchronized state, called 'SynchronizedMotion', within the state diagram
- Synchronization of slave axis is done according to current position (camming) or current velocity (gearing) of master axis

Multi-Axis Motion Camming



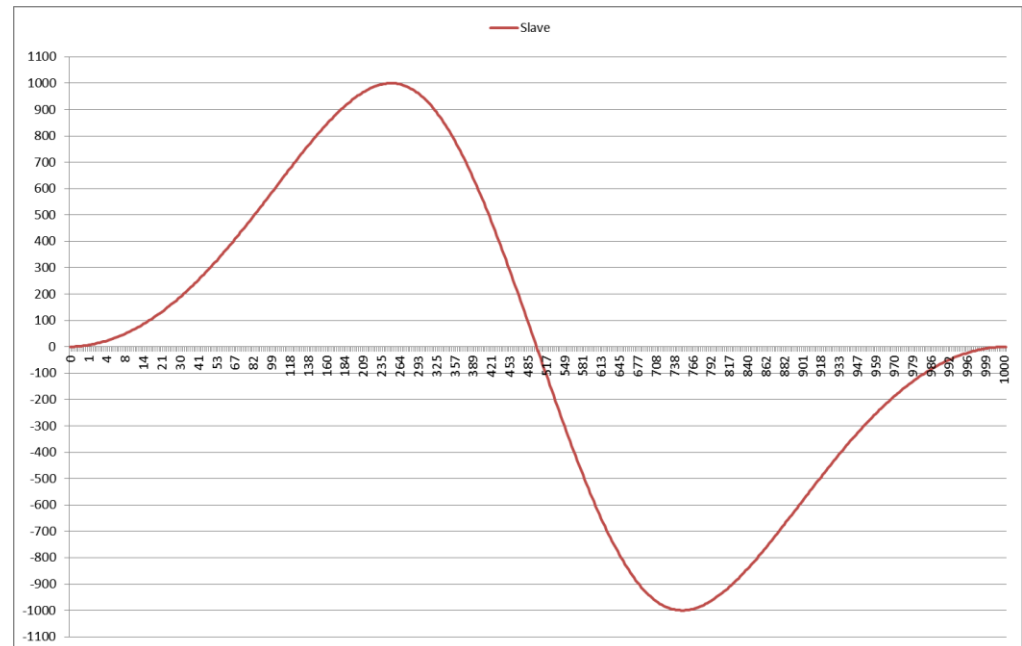
- **McCamTableSelect:** Function Block selects the CAM tables by setting the connections to the relevant tables
- **McCamIn:** Function Block engages the CAM
- **McCamOut:** Function Block disengages the Slave axis from the Master axis immediately

Multi-Axis Motion Camming: Example

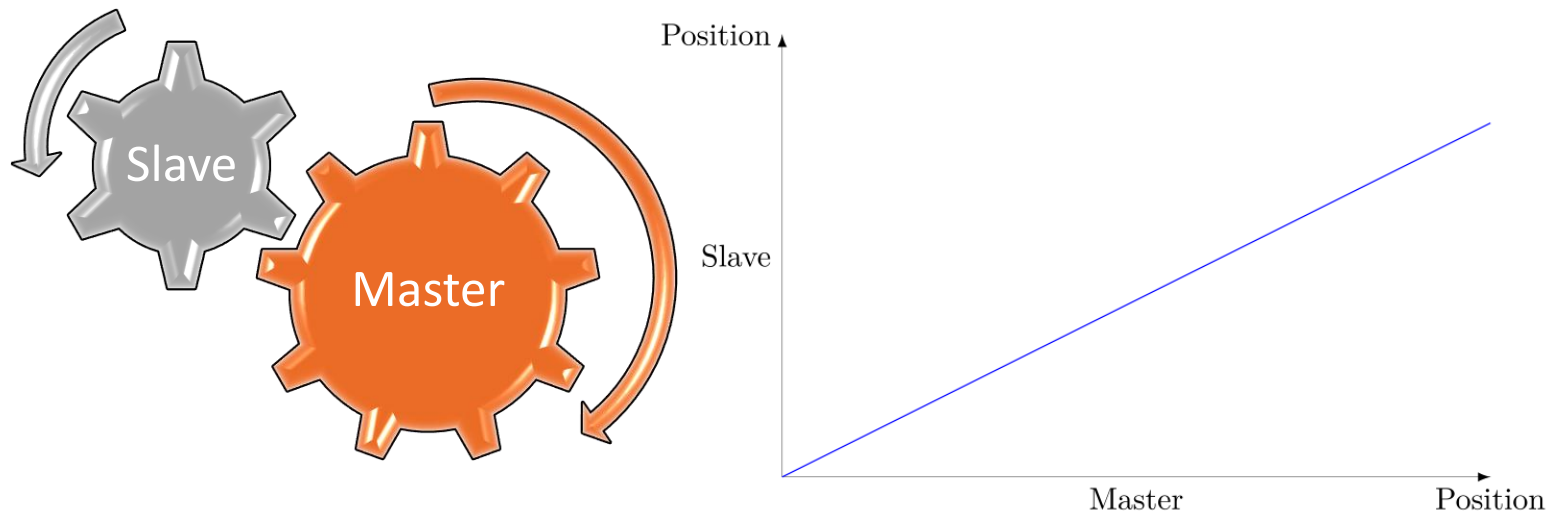
```
/* parametrization */
{
    EC_T_LREAL S_aCamPoints[][2] = {{0,0}, {125,707}, {250,1000}, {375,707}, {500,0},
                                     {625,-707}, {750,-1000}, {875,-707}, {1000,0}};
    MC_T_CAM_REF oCamTable;
    oCamTable.dwDegree = 3;
    oCamTable.dwNumOfElements = sizeof(S_aCamPoints) / sizeof(EC_T_LREAL) / 2;
    oCamTable.palfData = S_aCamPoints;
}

/* cyclic application */
{
    McCamTableSelect oCamTableSelect;
    McCamIn oCamIn;
    oCamTableSelect.Master = pMcMasterAxis;
    oCamTableSelect.Axis = pMcSlaveAxis;
    oCamTableSelect.Execute = EC_TRUE;
    oCamIn.Master = pMcMasterAxis;
    oCamIn.Slave = pMcSlaveAxis;

    while (EC_TRUE)
    {
        oCamTableSelect();
        oCamIn.Execute = oCamTableSelect.Done;
        oCamIn.CamTableID = oCamTableSelect.CamTableID;
        oCamIn();
        if (oCamIn.InSync)
        {
            /* move master axis */
        }
    }
}
```

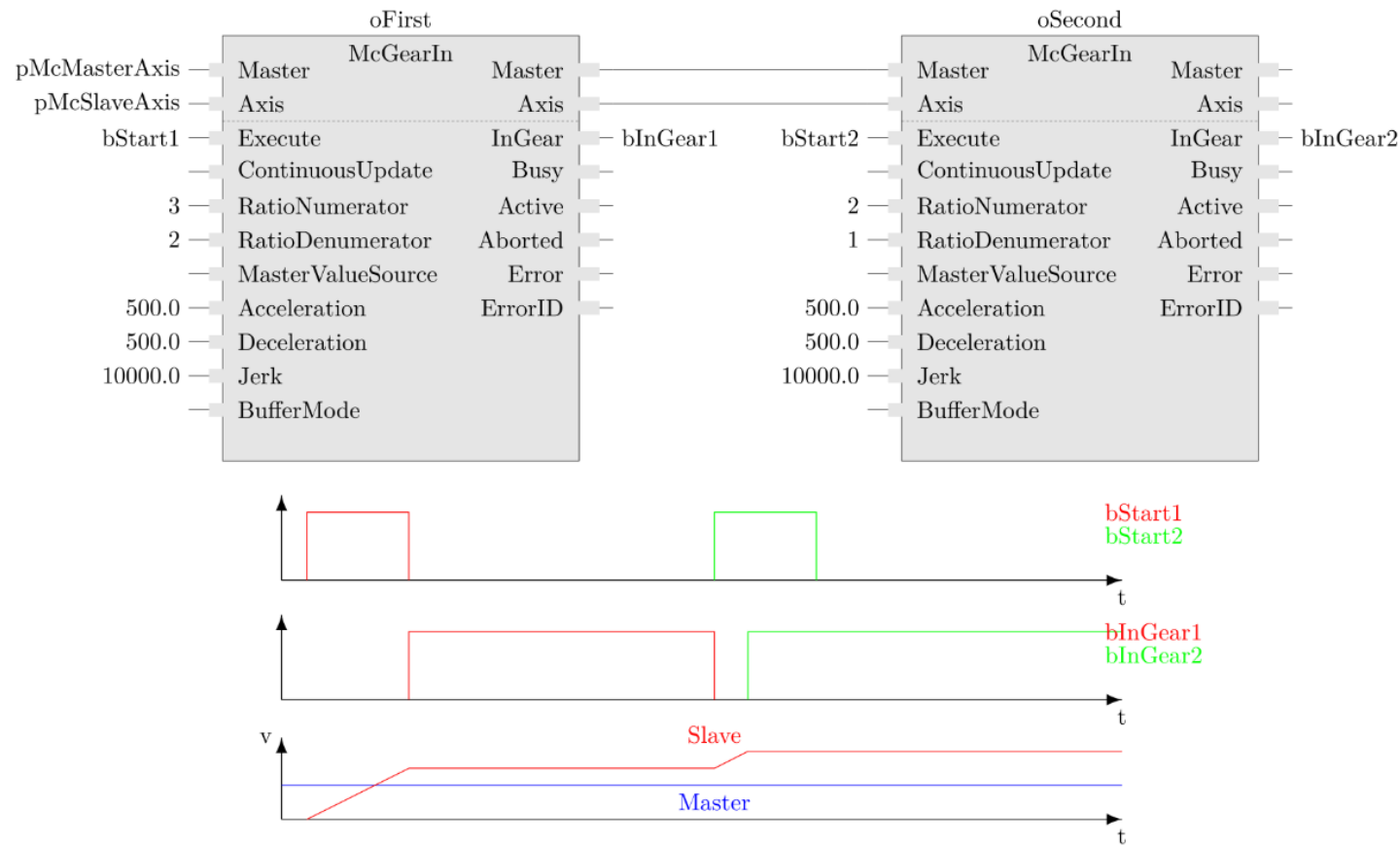


Multi-Axis Motion Gearing



- **McGearIn:** Function Block commands a ratio between the VELOCITY of the Slave and Master axis
- **McGearOut:** Function Block disengages the Slave axis from the Master axis

Multi-Axis Motion Gearing: Example (1)



Multi-Axis Motion Gearing: Example (2)

```
/* parametrization */
{
    McGearIn oFirst;
    McGearIn oSecond;
    oFirst.Master = oSecond.Master = pMcMasterAxis;
    oFirst.Axis = oSecond.Slave = pMcSlaveAxis;
    oFirst.RatioNumerator = 3;
    oFirst.RatioDenominator = 2;
    oFirst.Acceleration = oSecond.Acceleration = 500.0;
    oFirst.Deceleration = oSecond.Deceleration = 500.0;
    oFirst.Jerk = oSecond.Jerk = 10000.0;
    oSecond.RatioNumerator = 2;
    oSecond.RatioDenominator = 1;
}

/* cyclic application */
while (EC_TRUE)
{
    mcRefreshAxisInputs(0, 0, pbyPdIn);

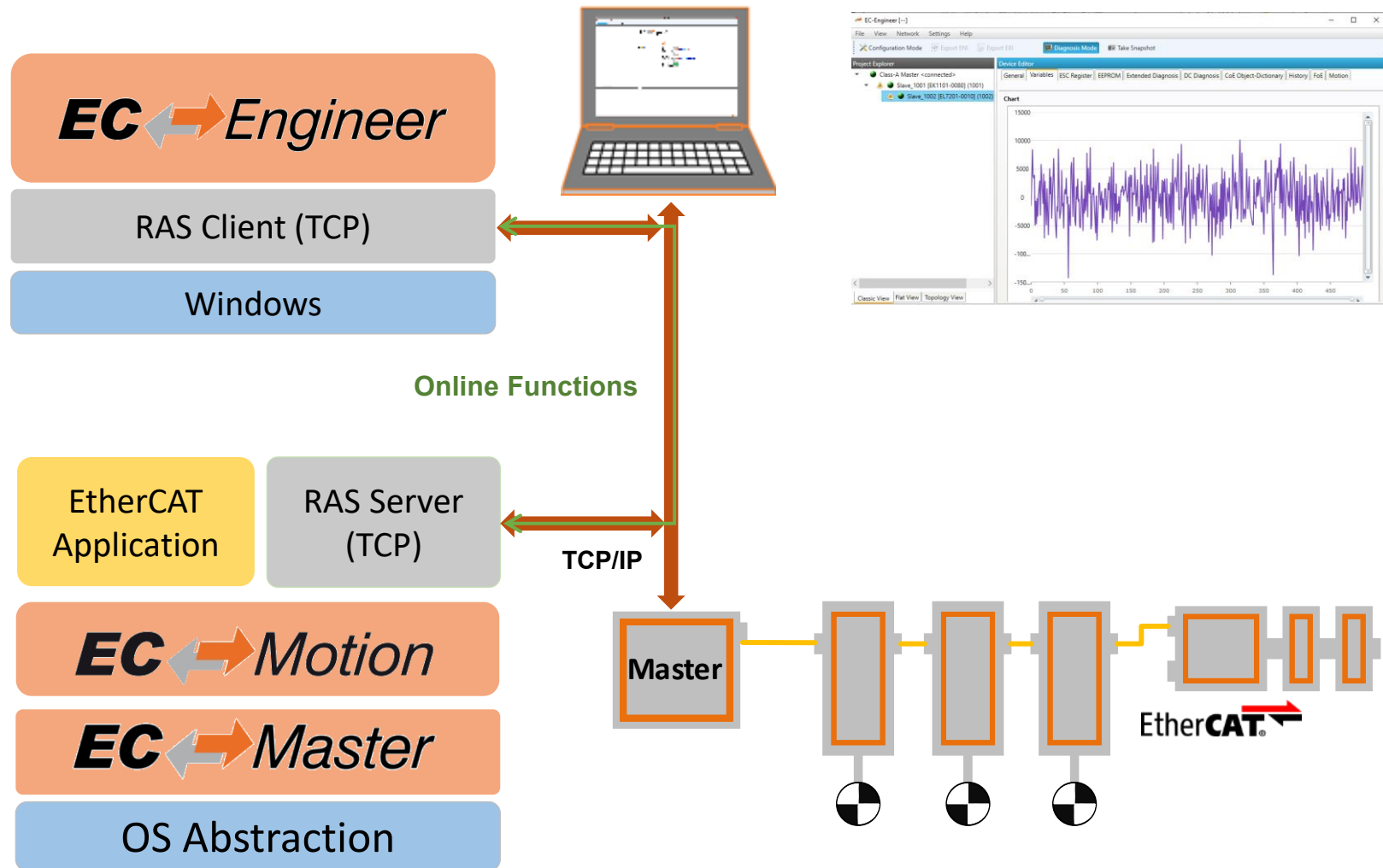
    oFirst.Execute = bStart1;
    oFirst();
    oSecond.Execute = bStart2;
    oSecond();

    mcCyclicTask(0, 0);
    mcRefreshAxisOutputs(0, 0, pbyPdOut);
}
```



Diagnosis tool and example application
EcMasterDemoMotionAdvanced

Tool EC-Engineer



EC-Engineer

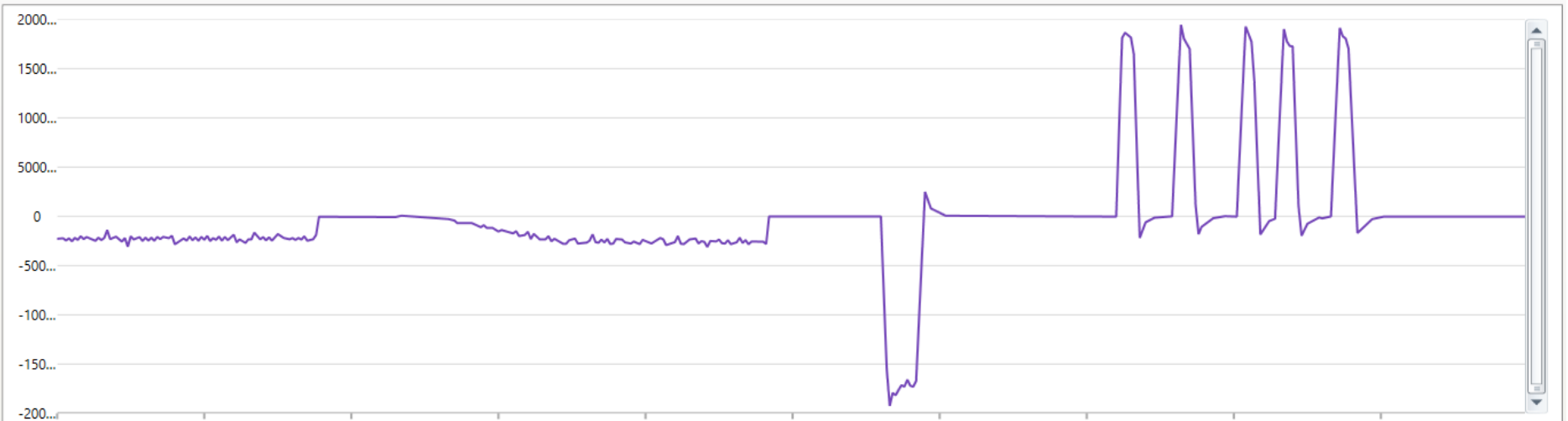
Monitoring actual velocity

Variables

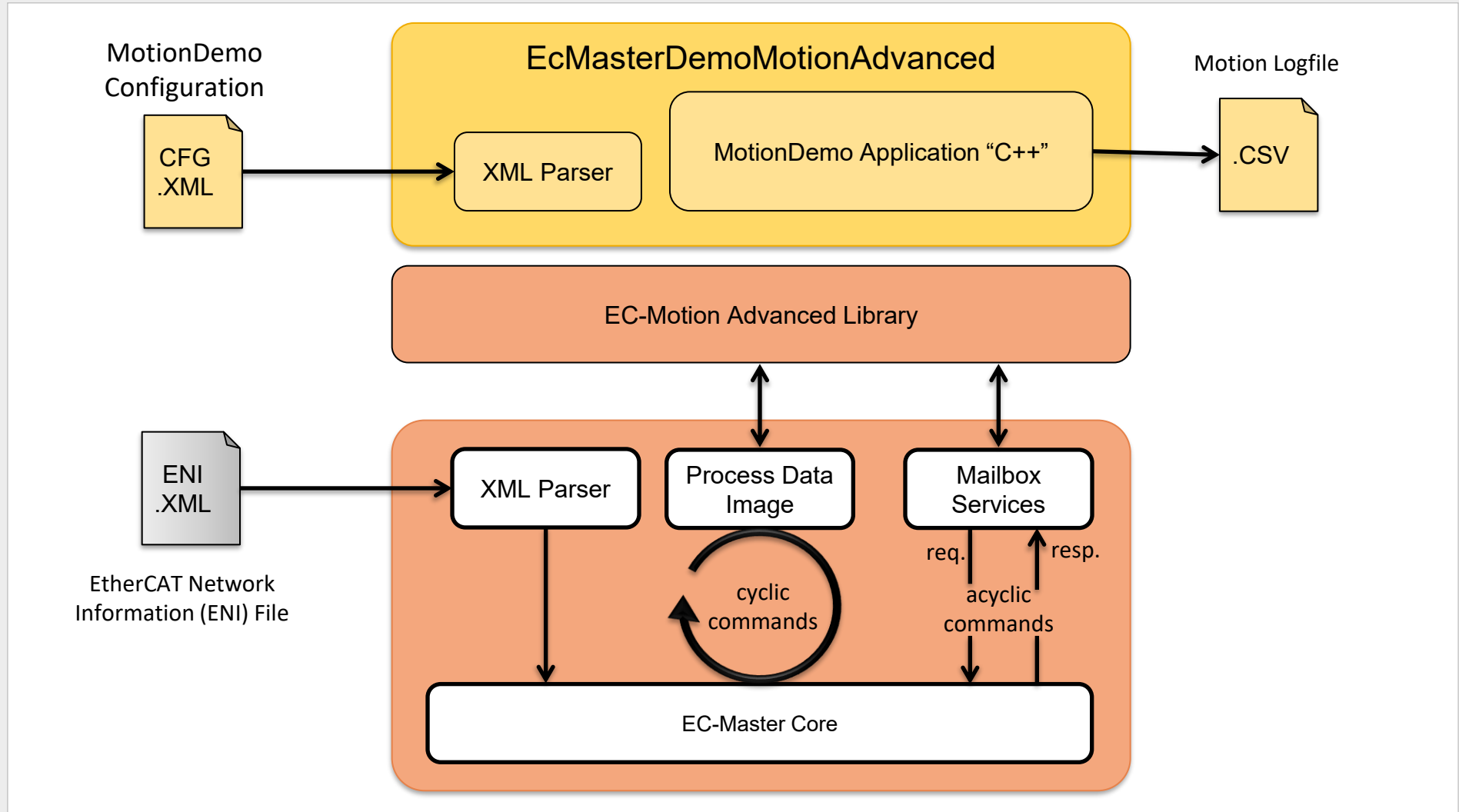
Name	Datatype	Offset	Size	Value	Forced
Slave_1002 [EL7201-0010].FB Position.Position	UDINT	IN :	0.0 4.0	74999952	☐
Slave_1002 [EL7201-0010].DRV Statusword.Statusword	UINT	IN :	4.0 2.0	4135	☐
Slave_1002 [EL7201-0010].DRV Velocity actual value.Velocity actual value	DINT	IN :	6.0 4.0	-2129	☐
Slave_1002 [EL7201-0010].DRV Controlword.Controlword	UINT	OUT :	0.0 2.0	15	☐
Slave_1002 [EL7201-0010].DRV Target velocity.Target velocity	DINT	OUT :	2.0 4.0	0	☐
Slave_1002 [EL7201-0010].DRV Target position.Target position	UDINT	OUT :	6.0 4.0	75000000	☐

Add to watch list

Chart



Example EcMasterDemoMotionAdvanced Software Architecture



EC-Engineer

Create MotionDemo configuration file

The screenshot displays the EC-Engineer software interface. The top menu bar includes File, View, Network, Settings, and Help. Below the menu is a toolbar with buttons for Configuration Mode (active), Export ENI, Export EXI, and Diagnosis Mode. The left sidebar shows the Project Explorer with a tree view containing 'Class-A Master' and 'Slave_1001 [Accelerometer]'. The main window is titled 'Device Editor' and has tabs for General, Modules, PDO Mapping, Variables, Advanced Options, Distributed Clock, Init Commands, CoE Object-Dictionary, Sync Units, and Motion (selected). The Motion tab shows 'Slave Settings' for two axes. Axis 1 and Axis 2 both have checkboxes checked. The settings for each axis are as follows:

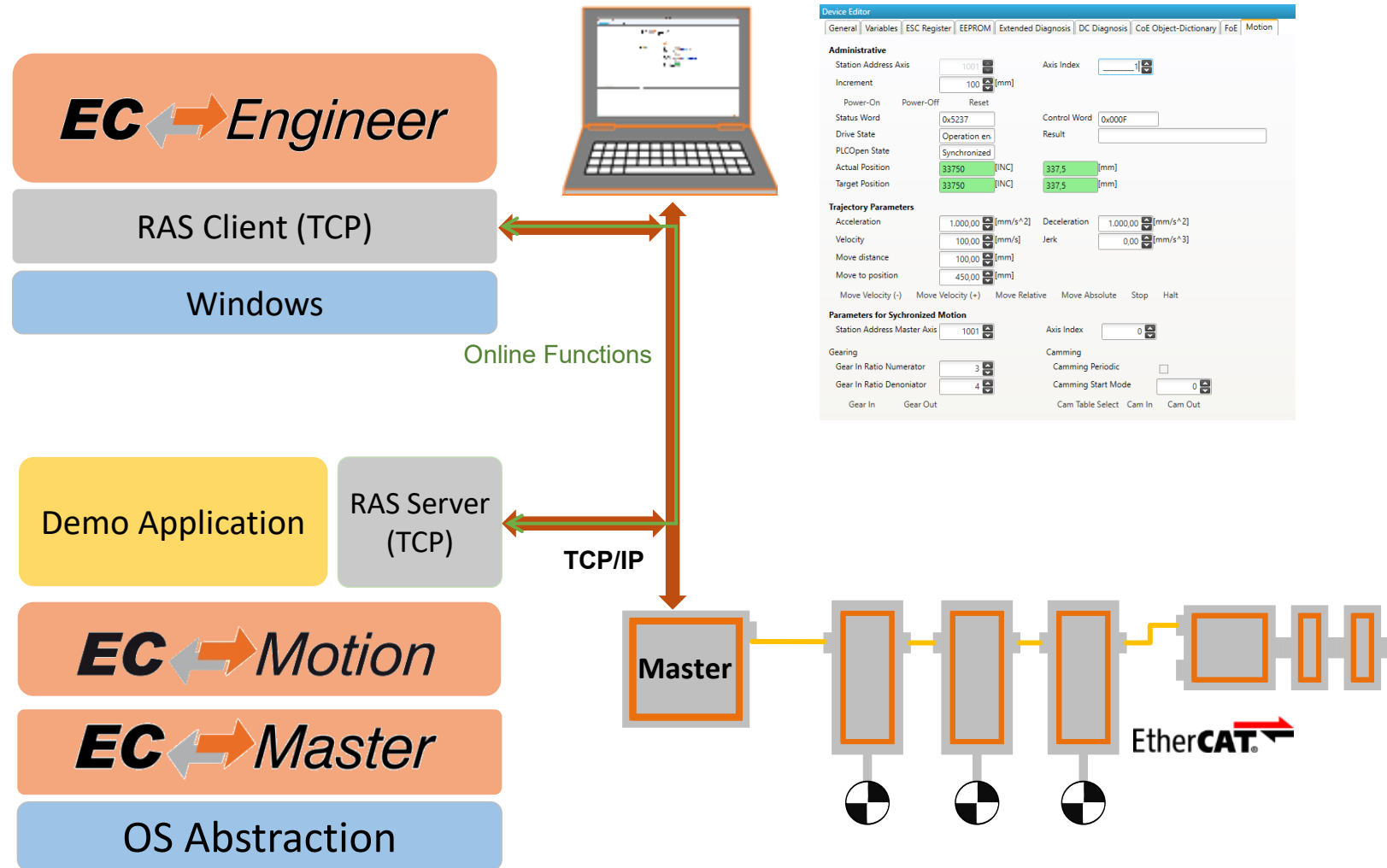
Setting	Axis 1	Axis 2
Mode of Operation	8	8
Increments per mm	1000	1000
Increment Factor	0	0
Controlword Object	0x6040 (Dec) [Hex]	0x6840 (Dec) [Hex]
Statusword Object	0x6041 (Dec) [Hex]	0x6841 (Dec) [Hex]
Position Actual Value Object	0x6064 (Dec) [Hex]	0x6864 (Dec) [Hex]
Target Position Object	0x607A (Dec) [Hex]	0x687A (Dec) [Hex]
Target Velocity Object	0x60FF (Dec) [Hex]	0x68FF (Dec) [Hex]
Modes of operation Object	0x6060 (Dec) [Hex]	0x6860 (Dec) [Hex]

Below the Slave Settings, there is a 'Common Settings' section with the following options:

- Enable RAS: ☒ Port: 6000
- Use Aux Clock: ☐
- CPU affinity: 1
- Link Layer: -winpcap 192.168.1.1
- Verbosity level: 0
- ENI Path: enter Path
- Performance Measurement: ☐

At the bottom right of the Common Settings section is a button labeled 'Export Demo Config'. To the right of the software interface, a large blue arrow points from the 'Export Demo Config' button to a yellow document icon labeled 'CFG .XML'.

EC-Engineer Remote control EcMasterDemoMotionAdvanced



EC-Engineer

Remote control EcMasterDemoMotionAdvanced

Device Editor

General Variables ESC Register EEPROM Extended Diagnosis DC Diagnosis CoE Object-Dictionary FoE Motion

Administrative

Station Address Axis Axis Index

Increment [mm]

Power-On Power-Off Reset

Status Word Control Word

Drive State Result

PLCOpen State

Actual Position [INC] [mm]

Target Position [INC] [mm]

Trajectory Parameters

Acceleration [mm/s²] Deceleration [mm/s²]

Velocity [mm/s] Jerk [mm/s³]

Move distance [mm]

Move to position [mm]

Move Velocity (-) Move Velocity (+) Move Relative Move Absolute Stop Halt

Parameters for Synchronized Motion

Station Address Master Axis Axis Index

Gearing

Gear In Ratio Numerator

Gear In Ratio Denoniator

Gear In Gear Out

Camming

Camming Periodic ☐

Camming Start Mode

Cam Table Select Cam In Cam Out

- ✓ No extra Motion Controller hardware required for operating drives
 - ✓ Cost reduction
- ✓ Perfected integration with acontis MainDevice Stack EC-Master
 - ✓ Saves effort
- ✓ Out-of-the-box available for many operating systems and CPU architectures.
No adaption required, start right away
 - ✓ Saves time
- ✓ Single supplier for Motion Control solution and MainDevice Stack
 - ✓ Simplified implementation, less time spent on contracts and paperwork